Graph Representational Learning

Dr. Abhilasha Chaudhuri

Graph Representational Learning(GRL) : Introduction

- What is a graph?
  - Multi-relational Graphs:
    - Heterogeneous graph
    - Multiplex graph

    In multiplex graphs we assume that the graph can be decomposed in a set of k layers. Every node is assumed to belong to every layer, and each layer corresponds to a unique relation, representing the intralayer edge type for that layer.
  - Feature Information

    we also have attribute or feature information associated with a graph (e.g., a profile picture associated with a user in a social network)

Graph Representational Learning(GRL) : Introduction

Most often these are node-level attributes that we represent using a real valued matrix $X \in \mathbb{R}^{|V| \times m}$, where we assume that the ordering of the nodes is consistent with the ordering in the adjacency matrix.

In heterogeneous graphs we generally assume that each different type of node has its own distinct type of attributes.

Machine Learning on Graphs

- Machine learning is inherently a problem-driven discipline.
- We seek to build models that can learn from data in order to solve particular tasks, and machine learning models are often categorized according to the type of task they seek to solve:
  - Is it a supervised task
  - Or an unsupervised task
- Machine learning with graphs is no different, but the usual categories of supervised and unsupervised are not necessarily the most informative or useful when it comes to graphs

Machine Learning on Graphs

Machine learning tasks on graph data:

- Node Classification
- Relation Prediction
- Clustering and community detection
- Graph classification, regression and clustering

Node Classification

- Suppose we are given a large social network dataset with millions of users, but we know that a significant number of these users are actually bots. Identifying these bots could be important.
- This is a classic example of node classification, where the goal is to predict the label y_u - which could be a type, category, or attribute -associated with all the nodes $u \in V$, when we are only given the true labels on a training set of nodes $V_{train} \subset V$.
- Examples of node classification beyond social networks include classifying the function of proteins in the interactome[Hamilton et al., 2017b]
- Classifying the topic of documents based on hyperlink or citation graphs [Kipf and Welling, 2016a]

Node Classification

- At first glance, node classification appears to be a straightforward variation of standard supervised classification, but there are in fact important differences.
- The most important difference is that the nodes in a graph are not independent and identically distributed (i.i.d.).
- Usually, when we build supervised machine learning models, we assume that each datapoint is statistically independent from all the other datapoints; otherwise, we might need to model the dependencies between all our input points.
- We also assume that the datapoints are identically distributed; otherwise, we have no way of guaranteeing that our model will generalize to new datapoints. Node classification completely breaks this i.i.d. assumption.
- Rather than modeling a set of i.i.d. datapoints, we are instead modeling an interconnected set of nodes

Node Classification

- In fact, the key insight behind many of the most successful node classification approaches is to explicitly leverage the connections between nodes.
- One particularly popular idea is to exploit **homophily**, which is the tendency for nodes to share attributes with their neighbors in the graph.
- For example, people tend to form friendships with others who share the same interests or demographics. Based on the notion of homophily we can build machine learning models that try to assign similar labels to neighboring nodes in a graph [Zhou et al., 2004]

Node Classification

- Beyond homophily there are also concepts such as **structural equivalence** [Donnat et al., 2018], which is the idea that nodes with similar local neighborhood structures will have similar labels, as well as **heterophily**, which presumes that nodes will be preferentially connected to nodes with different labels
- When we build node classification models, we want to exploit these concepts and model the relationships between nodes, rather than simply treating nodes as independent datapoints

Relation Prediction

- Node classification is useful for inferring information about a node based on its relationship with other nodes in the graph.
- But what about cases where we are missing this relationship information? What if we know only some of protein-protein interactions that are present in a given cell, but we want to make a good guess about the interactions we are missing?
- Can we use machine learning to infer the edges between nodes in a graph?
- This task goes by many names, such as link prediction, graph completion, and relational inference, depending on the specific application domain. We will simply call it **relation prediction**.

Clustering and community detection

- Both node classification and relation prediction require inferring missing information about graph data, and in many ways, those two tasks are the graph analogues of supervised learning.
- **Community detection**, on the other hand, is the graph analogue of **unsupervised clustering**.
- The challenge of community detection is to infer latent community structures given only the input graph G = (V; E).
- The many real-world applications of community detection include uncovering functional modules in genetic interaction networks.
- Uncovering fraudulent groups of users in financial transaction networks.

Graph classification, regression and clustering

- The final class of popular machine learning applications on graph data involve classification, regression, or clustering problems **over entire graphs**.
- For instance, given a graph representing the structure of a molecule, we might want to build a regression model that could predict that molecule's toxicity or solubility [Gilmer et al., 2017].
- Or, we might want to build a classification model to detect whether a computer program is malicious by analyzing a graph-based representation of its syntax and dataflow [Li et al., 2019].
- In these graph classification or regression applications, we seek to learn over graph data, but instead of making predictions over the individual components of a single graph (i.e., the nodes or the edges), we are instead given a dataset of multiple different graphs and our goal is to make independent predictions specific to each graph.
- In the related task of graph clustering, the goal is to learn an unsupervised measure of similarity between pairs of graphs.

Background and Traditional Approaches

Background and Traditional Approaches

- What kinds of methods were used for machine learning on graphs prior to the advent of modern deep learning approaches?

1. Graph Statistics and Kernel Methods
   A. Node-level statistics and features: Node degree, Node centrality, clustering coefficients, closed triangles, motifs, ego graphs etc.
   B. Graph-level features and graph kernels: Bag of nodes, The Weisfeiler-Lehman kernel, Graphlets and path-based methods
2. Neighborhood Overlap Detection
   A. Local Overlap detection: Jaccard index, Salton overlap
   B. Global overlap detection: Katz index
3. Graph Laplacians and Spectral Methods

Node Embeddings

Node Embeddings

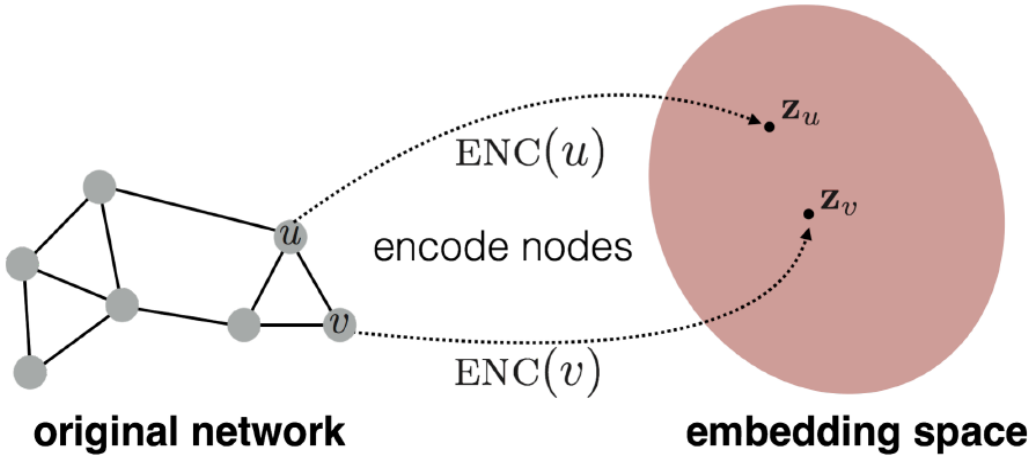


Figure 3.1: Illustration of the node embedding problem. Our goal is to learn an encoder (ENC), which maps nodes to a low-dimensional embedding space. These embeddings are optimized so that distances in the embedding space reflect the relative positions of the nodes in the original graph.

- The goal of **Node Embedding** method is to encode nodes as low-dimensional vectors that summarize their graph position and the structure of their local graph neighborhood.
- In other words, we want to project nodes into a latent space, where geometric relations in this latent space correspond to relationships (e.g., edges) in the original graph or network

Node Embeddings: An Encoder-Decoder Perspective

- In the encoder-decoder framework, we view the graph representation learning problem as involving two key operations.
- First, an encoder model maps each node in the graph into a low-dimensional vector or embedding.
- Next, a decoder model takes the low-dimensional node embeddings and uses them to reconstruct information about each node's neighborhood in the original graph.
- This idea is summarized in Figure 3.2.

Node Embeddings: An Encoder-Decoder Perspective

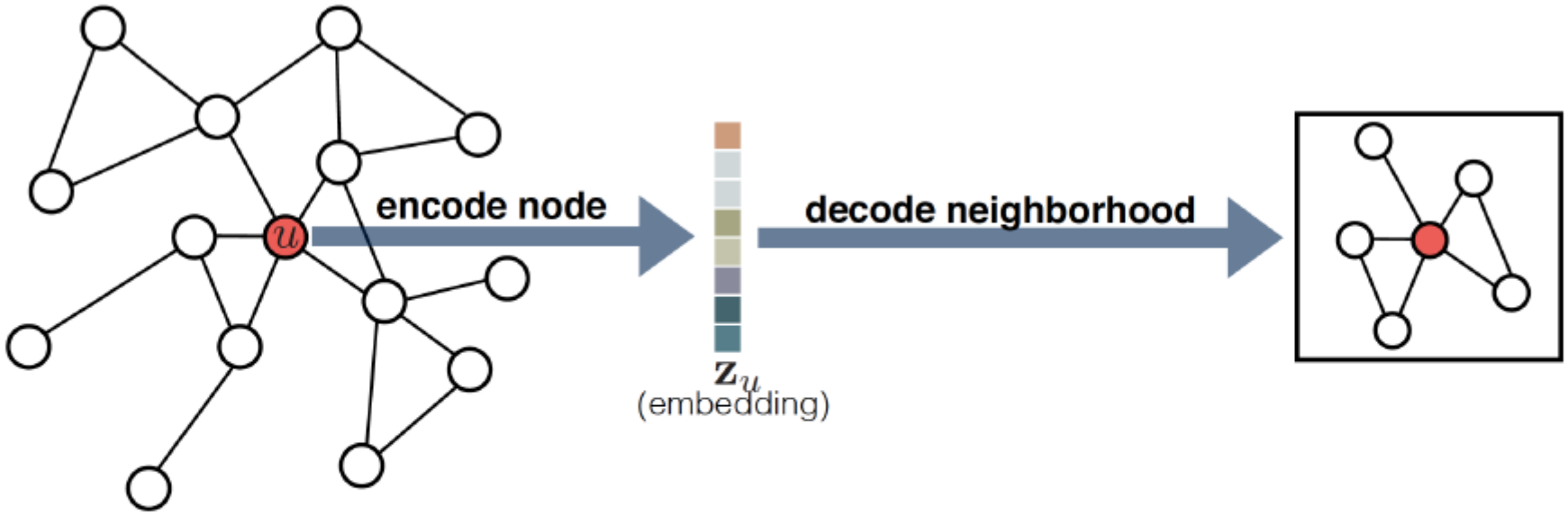


Figure 3.2: Overview of the encoder-decoder approach. The encoder maps the node u to a low-dimensional embedding $\mathbf{z}_u$. The decoder then uses $\mathbf{z}_u$ to reconstruct u's local neighborhood information.

Node Embeddings: An Encoder-Decoder Perspective

The Encoder:

Formally, the *encoder* is a function that maps nodes $v \in \mathcal{V}$ to vector embeddings $\mathbf{z}_v \in \mathbb{R}^d$ (where $\mathbf{z}_v$ corresponds to the embedding for node $v \in \mathcal{V}$). In the simplest case, the encoder has the following signature:

$$\text{ENC} : \mathcal{V} \rightarrow \mathbb{R}^d, \tag{3.1}$$

meaning that the encoder takes node IDs as input to generate the node embeddings. In most work on node embeddings, the encoder relies on what we call the *shallow embedding* approach, where this encoder function is simply an embedding lookup based on the node ID. In other words, we have that

$$\text{ENC}(v) = \mathbf{Z}[v], \tag{3.2}$$

where $\mathbf{Z} \in \mathbb{R}^{|\mathcal{V}| \times d}$ is a matrix containing the embedding vectors for all nodes and $\mathbf{Z}[v]$ denotes the row of $\mathbf{Z}$ corresponding to node v.

Node Embeddings: An Encoder-Decoder Perspective

The Decoder:

The role of the *decoder* is to reconstruct certain graph statistics from the node embeddings that are generated by the encoder. For example, given a node embedding $\mathbf{z}_u$ of a node u, the decoder might attempt to predict u's set of neighbors $\mathcal{N}(u)$ or its row $\mathbf{A}[u]$ in the graph adjacency matrix.

While many decoders are possible, the standard practice is to define *pairwise* decoders, which have the following signature:

$$\text{DEC} : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^+. \tag{3.3}$$

Pairwise decoders can be interpreted as predicting the relationship or similarity between pairs of nodes. For instance, a simple pairwise decoder could predict whether two nodes are neighbors in the graph.

Applying the pairwise decoder to a pair of embeddings $(\mathbf{z}_u,\mathbf{z}_v)$ results in the *reconstruction* of the relationship between nodes u and v. The goal is optimize the encoder and decoder to minimize the reconstruction loss so that

$$\text{DEC}(\text{ENC}(u), \text{ENC}(v)) = \text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \approx \mathbf{S}[u, v]. \tag{3.4}$$

Here, we assume that $\mathbf{S}[u, v]$ is a graph-based similarity measure between nodes.

Node Embeddings: An Encoder-Decoder Perspective

- Optimizing an Encoder-Decoder Model

To achieve the reconstruction objective (Equation 3.4), the standard practice is to minimize an empirical reconstruction loss $\mathcal{L}$ over a set of training node pairs $\mathcal{D}$:

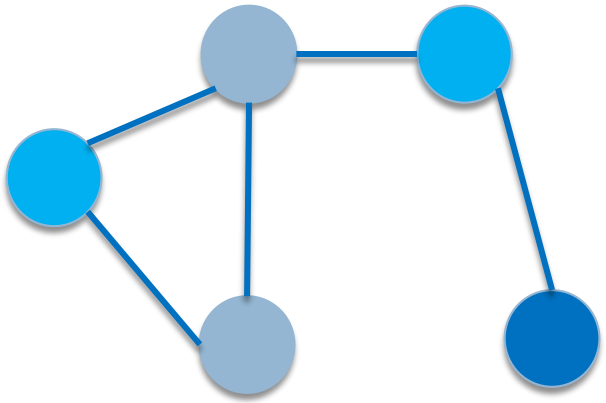
$$\mathcal{L} = \sum_{(u,v)\in\mathcal{D}} \ell\left(\text{DEC}(\mathbf{z}_u, \mathbf{z}_v), \mathbf{S}[u,v]\right), \tag{3.5}$$

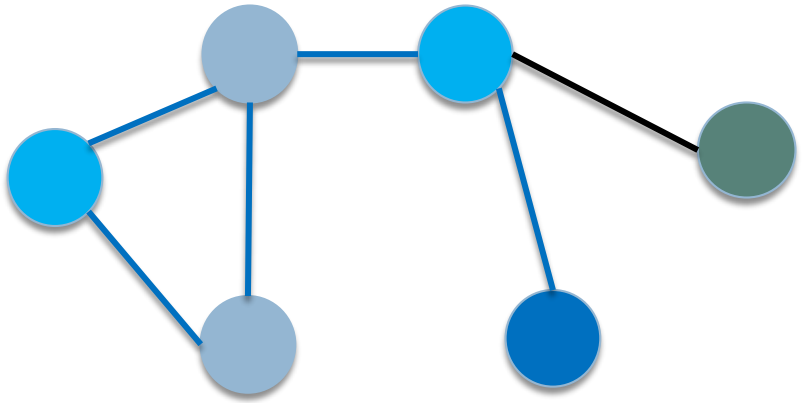
where $\ell : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$ is a loss function measuring the discrepancy between the decoded (i.e., estimated) similarity values $\text{DEC}(\mathbf{z}_u, \mathbf{z}_v)$ and the true values $\mathbf{S}[u,v]$. Depending on the definition of the decoder (DEC) and similarity function ($\mathbf{S}$), the loss function ℓ might be a mean-squared error or even a classification loss, such as cross entropy. Thus, the overall objective is to train the encoder and the decoder so that pairwise node relationships can be effectively reconstructed on the training set $\mathcal{D}$. Most approaches minimize the loss in Equation 3.5 using stochastic gradient descent [Robbins and Monro, 1951], but there are certain instances when more specialized optimization methods (e.g., based on matrix factorization) can be used.

Graph Neural Network

Properties of Graphs

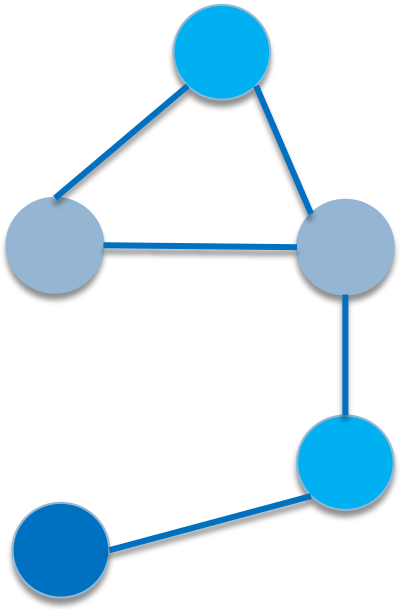
- Size and shape





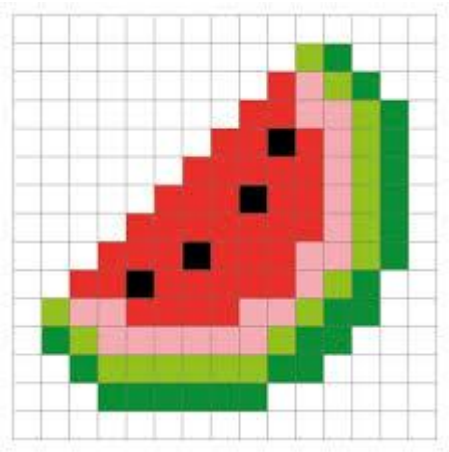
Properties of Graphs

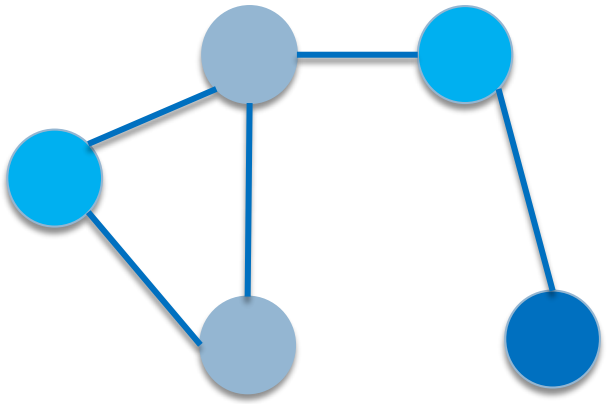
- Isomorphism
  - Permutation invariance
  - Adjacency matrix cannot be used as input



Properties of Graphs

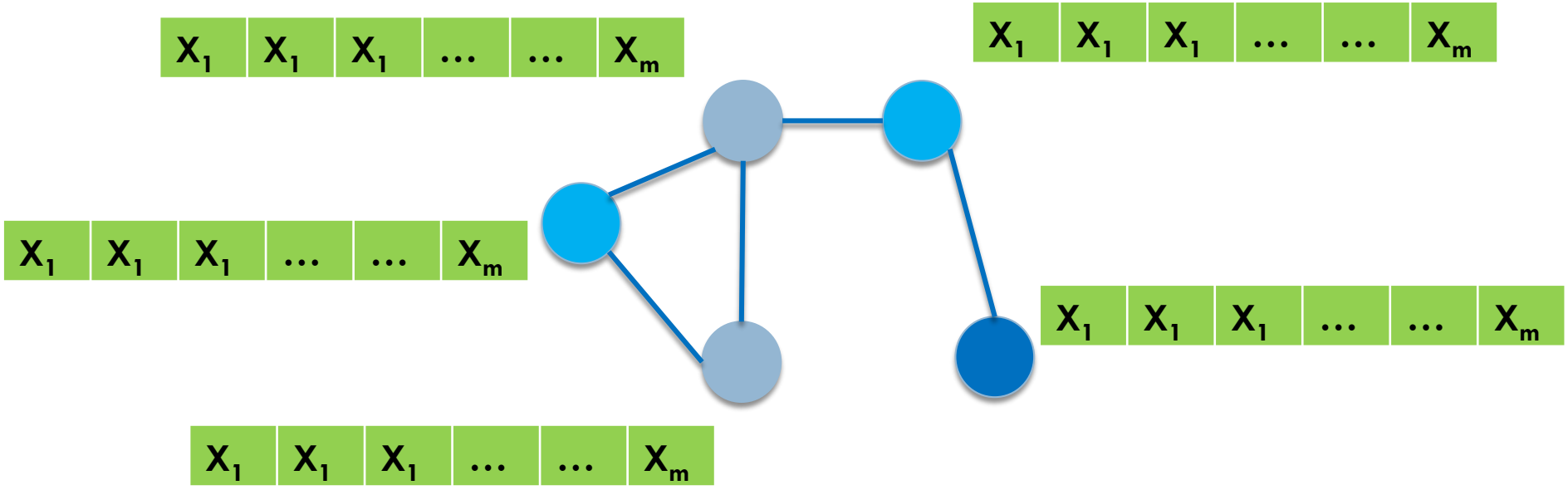
- Grid structure





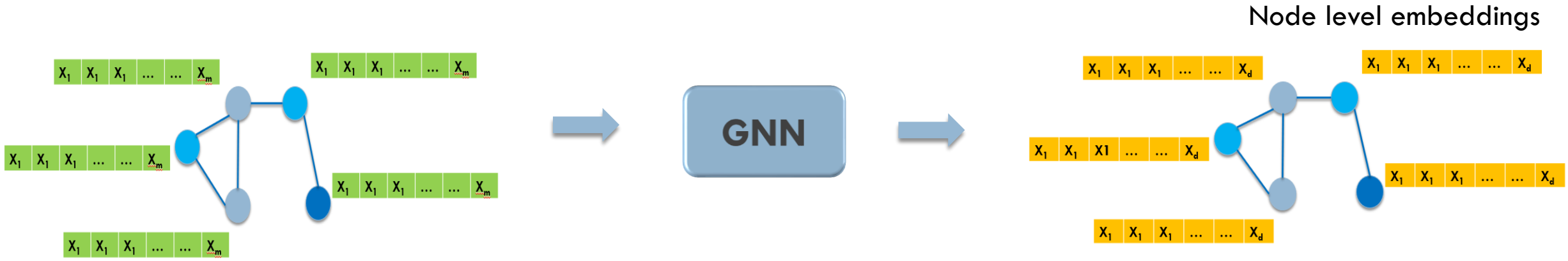
Non Euclidean Space

Graphs Representation



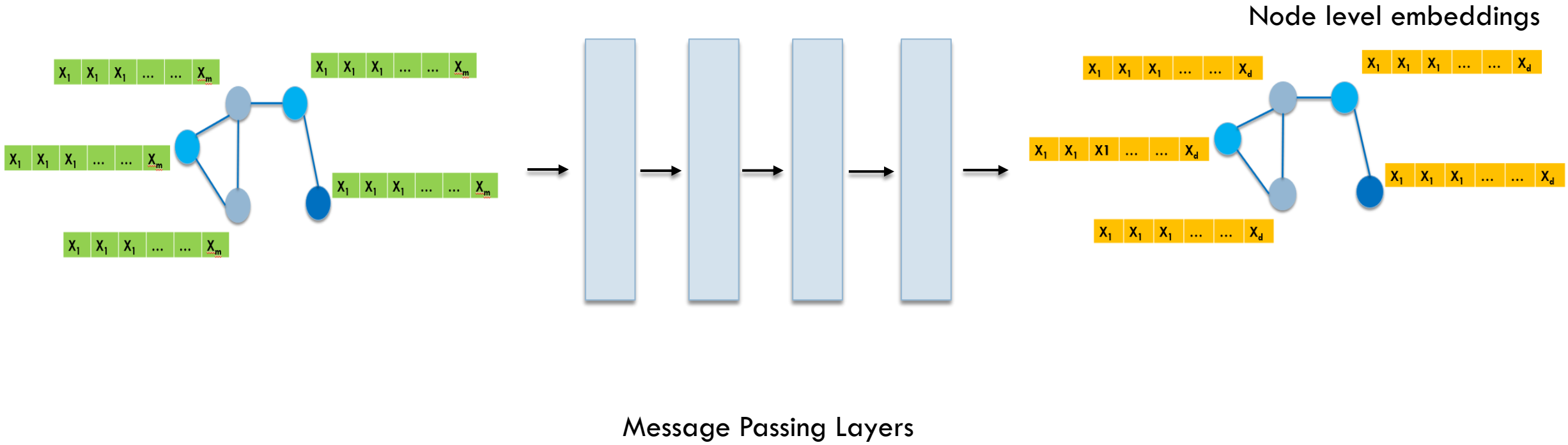
Fundamental Idea of GNN

- Learning a neural network *suitable representation* of graph data

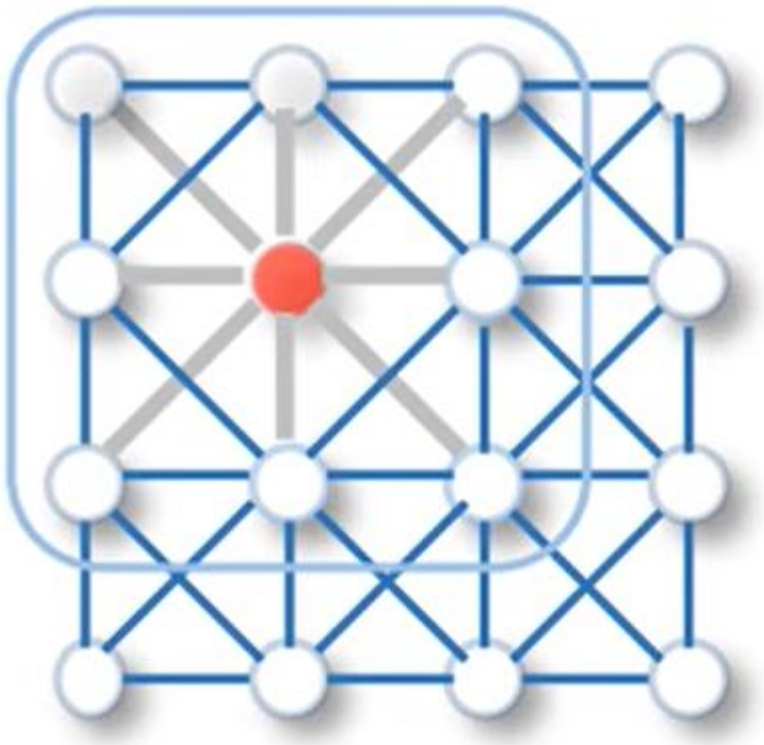


Fundamental Idea of GNN

- Learning a neural network *suitable representation* of graph data



What is happening in the message passing layers?



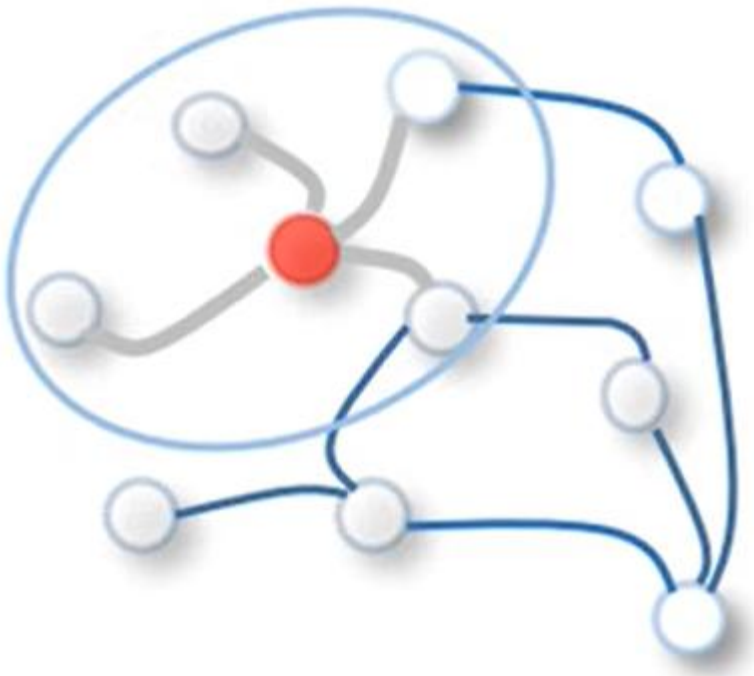
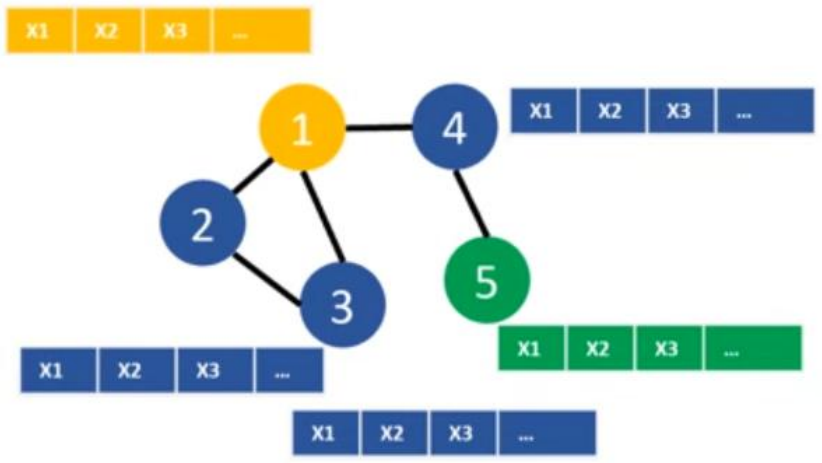


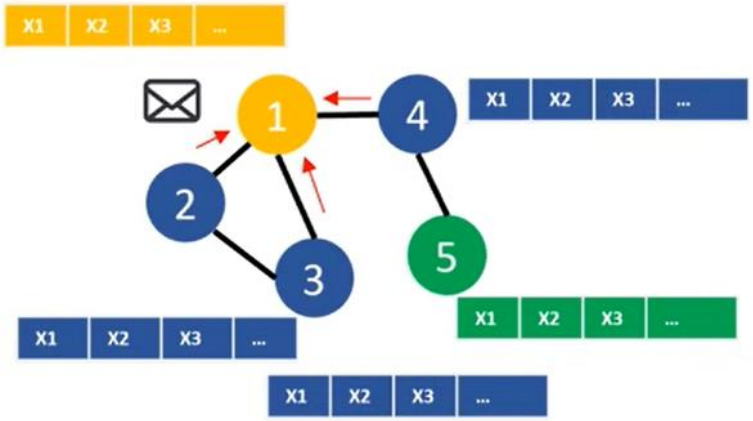
Image source: A comprehensive survey on Graph Neural Networks, Zonghan Wu et al., 2019

What is happening in the message passing layers?



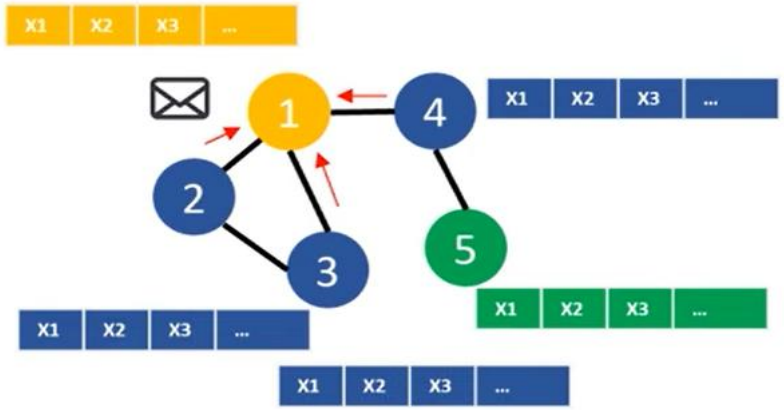
Let's assume this is our input graph with five nodes

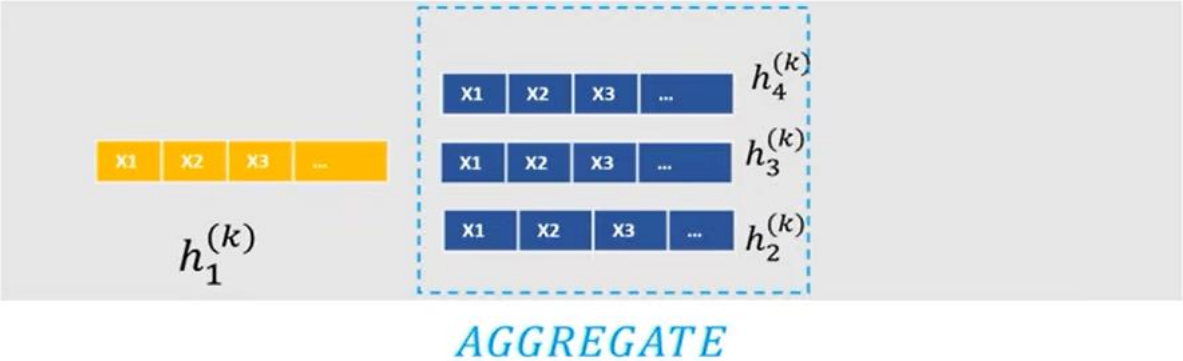
What is happening in the message passing layers?



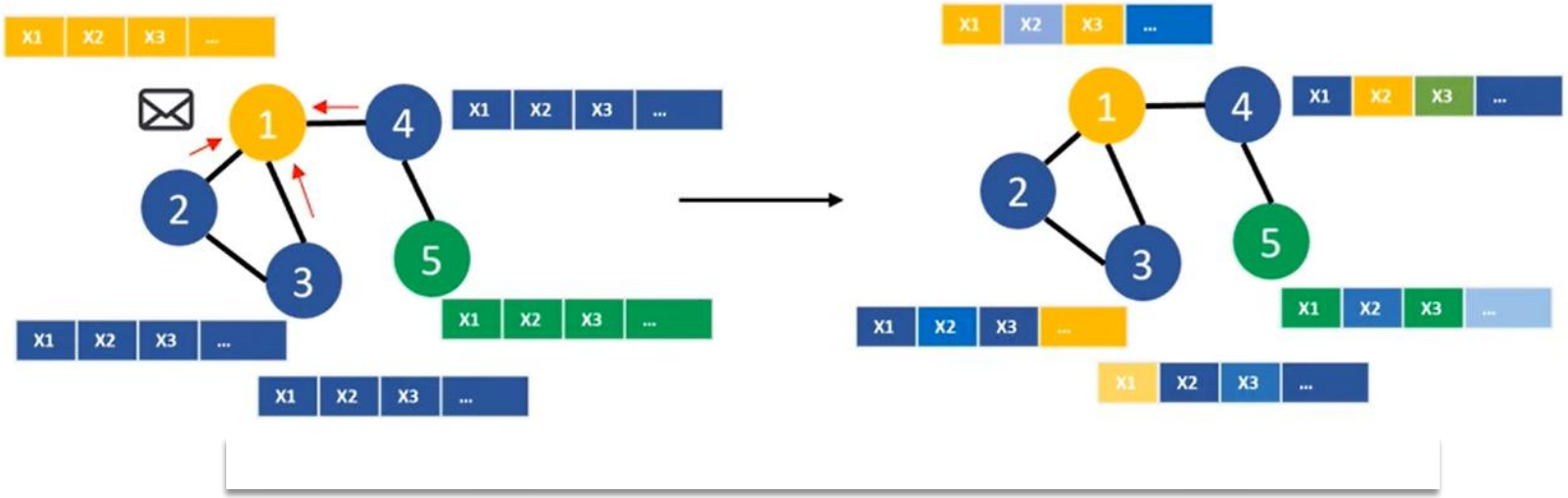
First layer of message passing

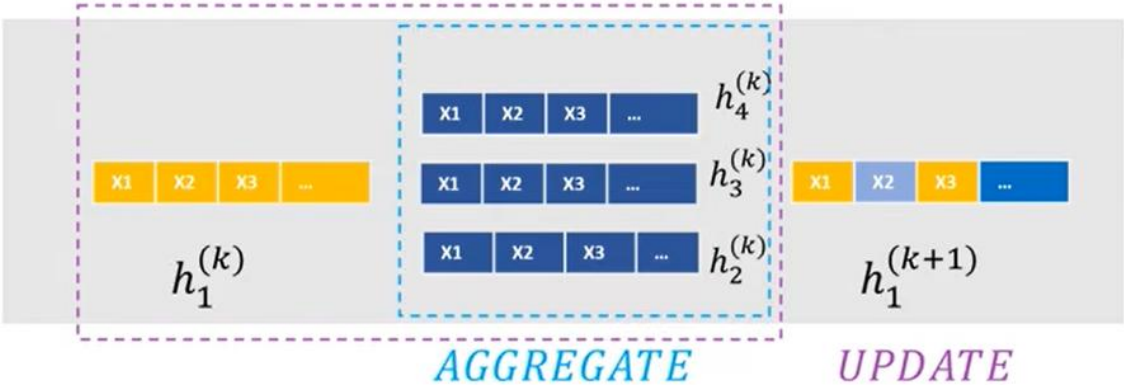
What is happening in the message passing layers?



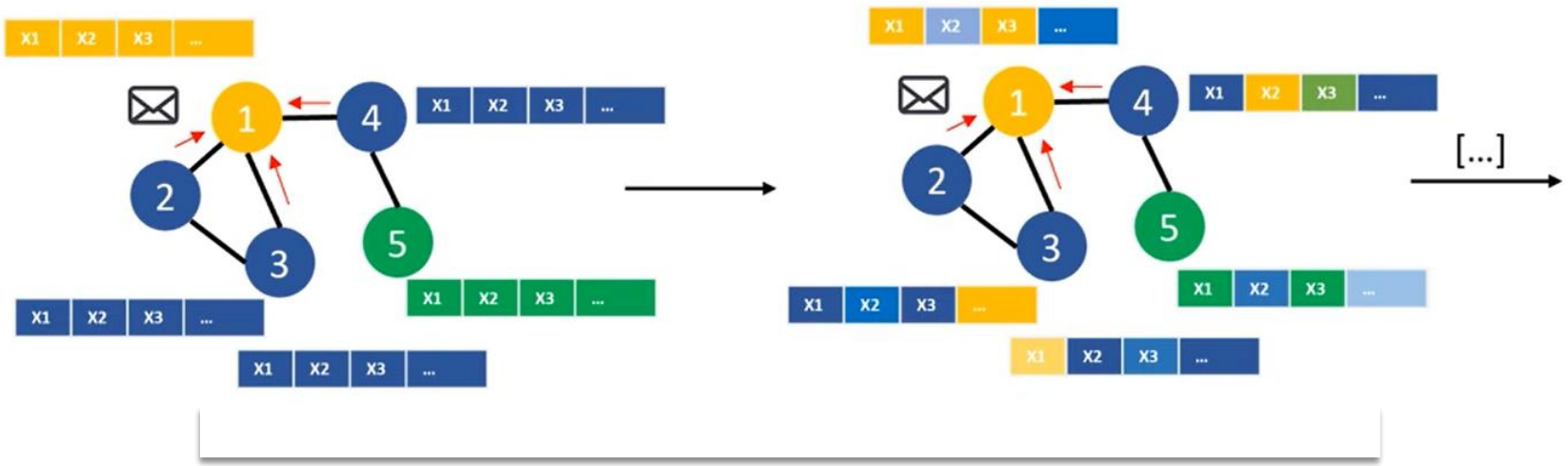


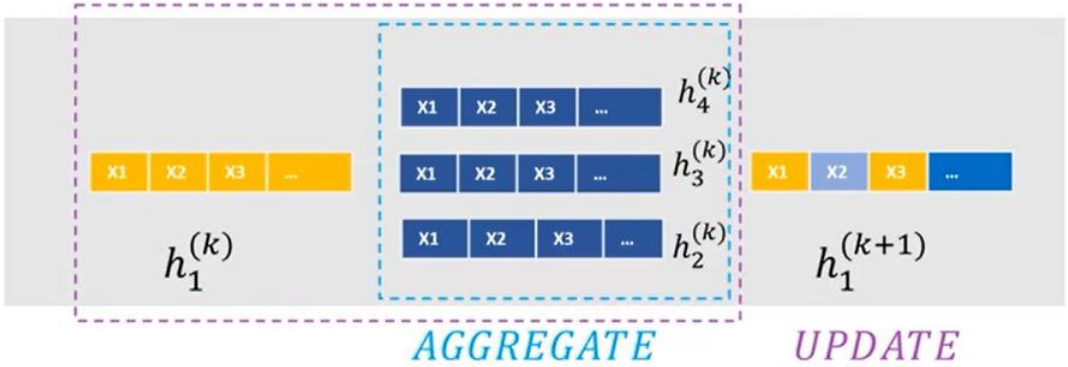
What is happening in the message passing layers?

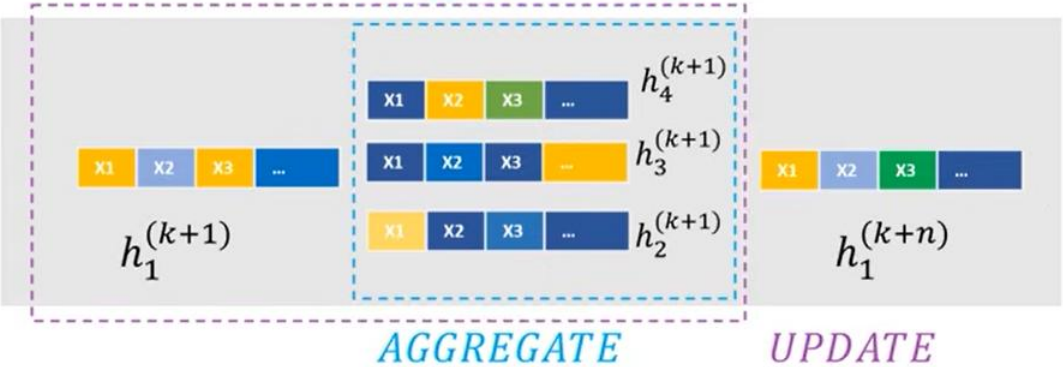




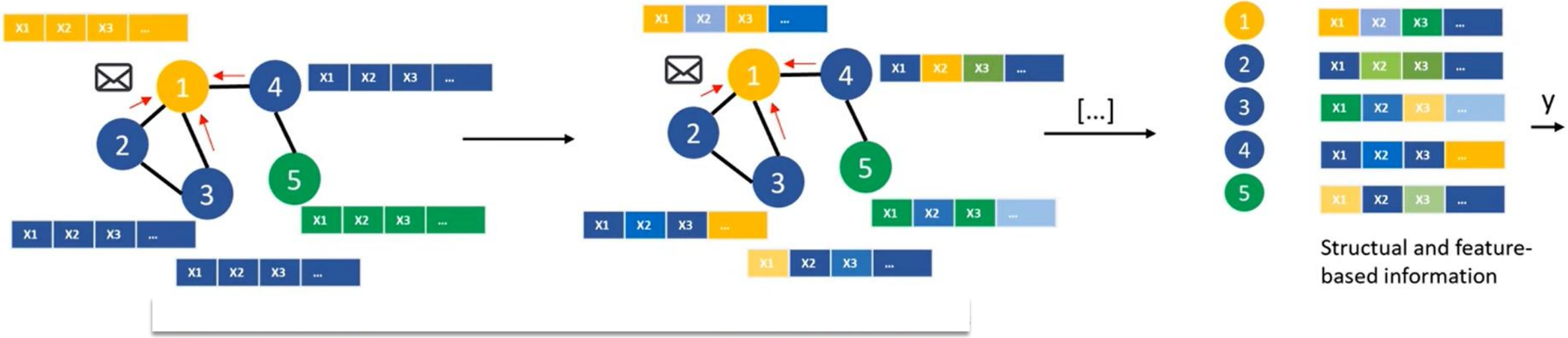
What is happening in the message passing layers?

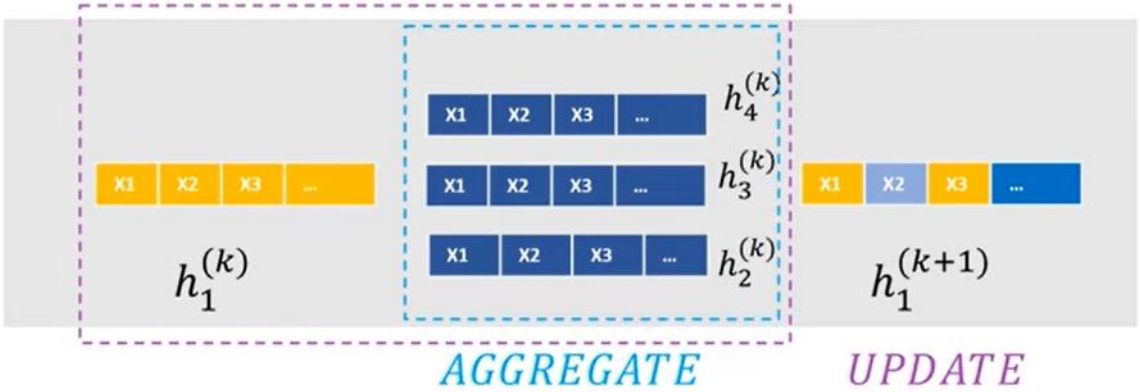


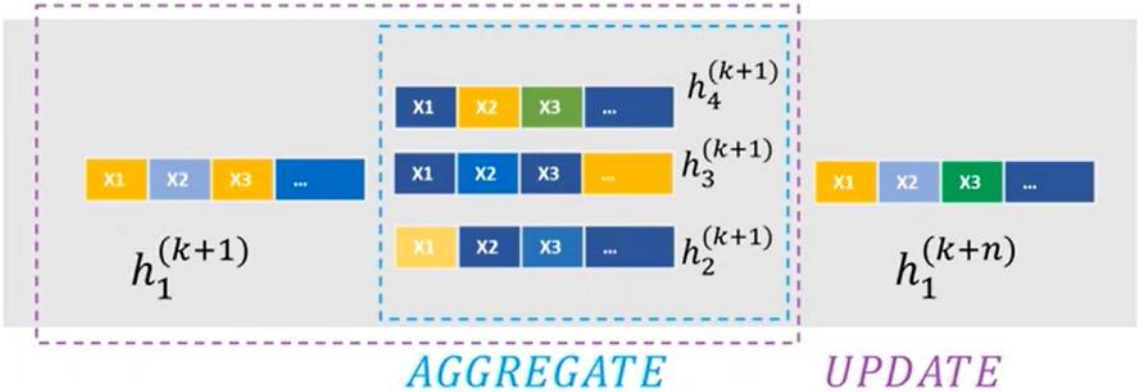




What is happening in the message passing layers?







Details of GNN

Details of GNN

- Refer to Section 5.1 and 5.2 from chapter 5 of the book named “Graph Representation Learning” (Author: William L. Hamilton)
- You need to study following topics:
- 5.1
  - 5.1.1
  - 5.1.2
  - 5.1.3
  - 5.1.4
- 5.2
  - 5.2.1
  - 5.2.2
  - 5.2.3

Variants of GNN

AGGREGATE
(permutation invariant)

UPDATE

Graph Convolutional Networks, Kipf and Welling [2016]

Self-loop

$$\mathbf{h}_v^{(k)} = \sigma\left(\mathbf{W}^{(k)} \sum_{v \in \mathcal{N}(u) \cup \{u\}} \frac{\mathbf{h}_v}{\sqrt{|\mathcal{N}(u)||\mathcal{N}(v)|}}\right)$$

Sum of normalized neighbor embeddings

Multi-Layer-Perceptron as Aggregator, Zaheer et al. [2017]

Aggregated message

$$\mathbf{m}_{\mathcal{N}(u)} = \boxed{\mathrm{MLP}_\theta}\left(\sum_{v \in N(u)} \mathrm{MLP}_\phi(\mathbf{h}_v)\right)$$

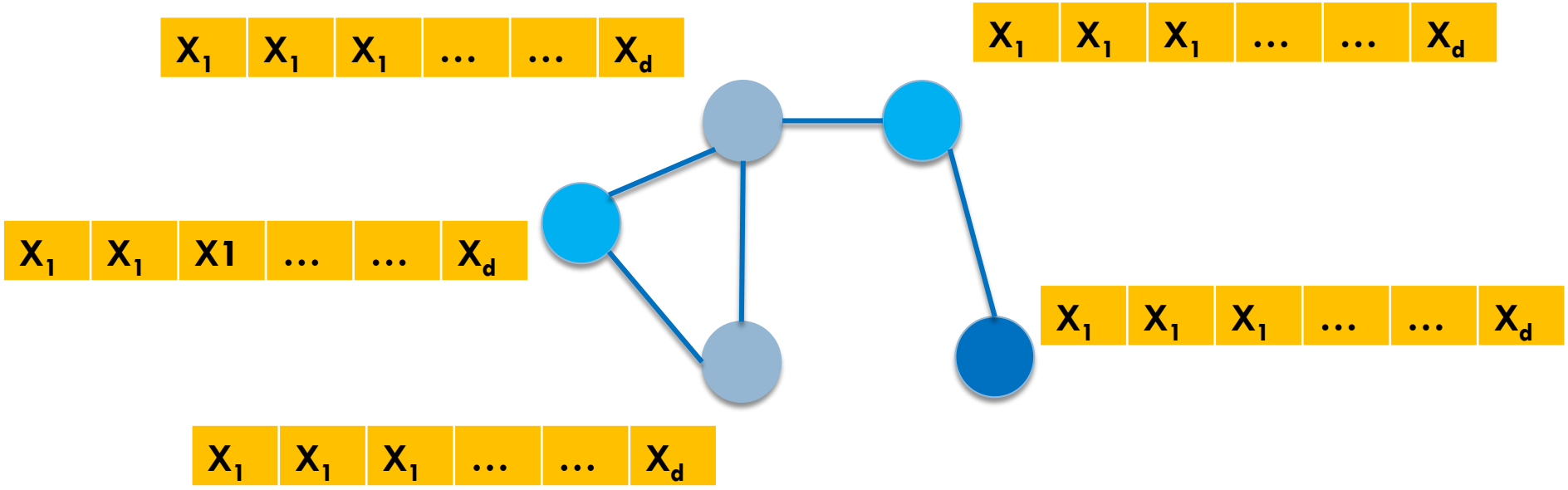
trainable!

Send states through a MLP

Graph Attention Networks, Veličković et al. [2017]

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \alpha_{u,v} \mathbf{h}_v$$

Attention weights

$$\alpha_{u,v} = \frac{\exp\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_v]\right)}{\sum_{v' \in \mathcal{N}(u)} \exp\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_{v'}]\right)}$$

Graphs Representation